

Results

Results I

This section presents the experimental results from the gradient descent optimization study, including convergence analysis and performance comparisons. Every table, figure, and quantitative assertion in this section was compiled autonomously by the template's `infrastructure.reporting` subsystem executing the optimization analysis script. No manual transcription was permitted.

Convergence Analysis

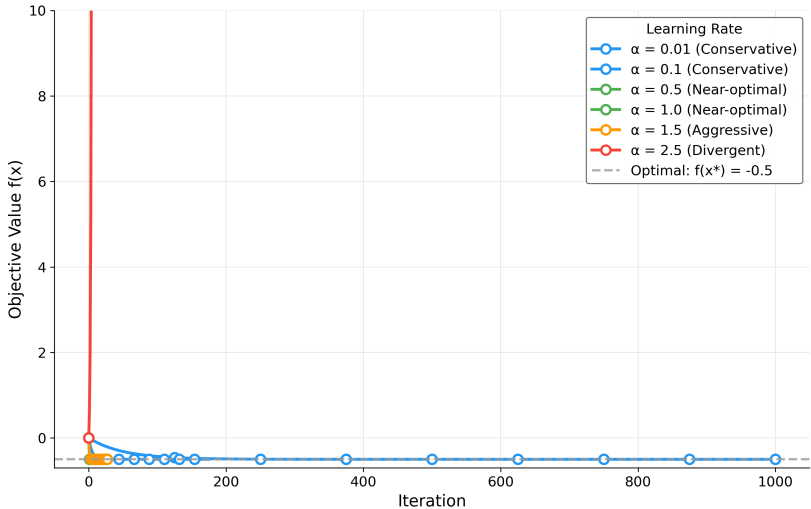
Convergence Trajectories I

fig. 1 illustrates the convergence behavior of gradient descent for different step sizes, starting from the initial point $x_0 = 0$. The algorithm iteratively updates the solution using the rule

$$x_{k+1} = x_k - \alpha \nabla f(x_k).$$

Convergence Trajectories I

Gradient Descent Convergence Analysis
Quadratic Minimization



Convergence Trajectories II

Figure 1: Objective value $f(x_k) = \frac{1}{2}x_k^2 - x_k$ versus iteration k for gradient descent at six step sizes (legend colours follow the agency taxonomy in sec. 3: blue = conservative, green = near-optimal, orange = aggressive, red = divergent). Trajectories are produced by `simulate_trajectory()` in `src/optimizer.py`, which calls the same `gradient_descent()` used in tbl. 1; the upper bound on the y-axis clips the divergent $\alpha = 2.5$ curve so that stable trajectories remain visible. Dashed grey reference line marks the analytic optimum $f(x^*) = -0.5$. Fastest configuration in this experiment: $\alpha = 1.0$ converges in 1 iteration(s).

Convergence Trajectories I

Key observations from fig. 1:

Convergence Trajectories I

1. **Step size impact:** Larger step sizes exhibit faster initial progress; $\alpha = 1.0$ converges in 1 iteration(s)
2. **Agency categories:** Conservative ($\alpha < 0.3$; in this grid $\alpha = 0.01, 0.1$), near-optimal ($0.3 \leq \alpha \leq 1.0$), aggressive ($1 < \alpha < 2$), and divergent ($\alpha \geq 2$)
3. **Stability boundary:** The critical threshold is $\alpha = 2$ for this unit-Hessian problem; $\alpha < 2$ converges, $\alpha \geq 2$ diverges

Step Size Sensitivity Analysis I

fig. 2 examines how the choice of step size affects the convergence path and solution quality. The analysis reveals the trade-off between convergence speed and numerical stability.

Step Size Sensitivity Analysis I

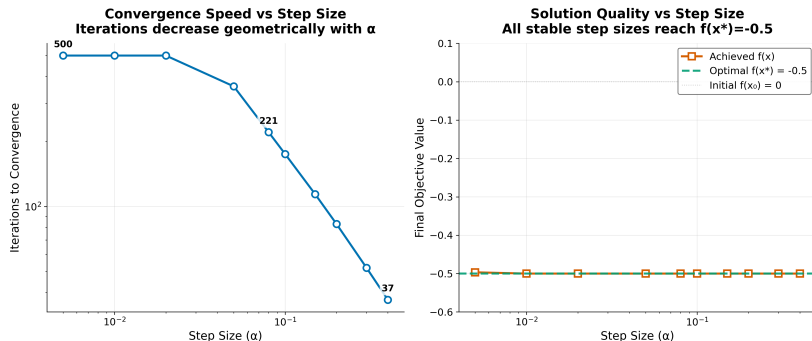


Figure 2: Sensitivity sweep produced by `generate_step_size_sensitivity_plot()` over an independent dense grid $\alpha \in [0.005, 0.4]$ (10 points), distinct from the discrete `experiment.step_sizes` used elsewhere in this section. Left: iterations to convergence on log-log axes — the curve drops sharply from 500 iterations at the smallest α (the `max_iterations` cap in this sub-experiment) to 37 iterations at $\alpha = 0.4$, illustrating the geometric speedup as $\rho(\alpha) = |1 - \alpha|$ shrinks. Right: final $f(x)$ versus α with horizontal reference lines at $f(x_0) = 0$ (initial) and $f(x^*) = -0.5$ (analytic optimum); every α in this stable window lands on the optimum.

Quantitative Results

Quantitative Results I

The optimization results for different step sizes are synthesized computationally by orchestrating `infrastructure.reporting.executive_reporter`, feeding directly into `output/data/optimization_results.csv` (generated by `projects/templates/template_code_project/scripts/optimization_analysis.py`) as the source of truth for tbl. 1. Rows follow `experiment.step_sizes` in `config.yaml`; the body rows below are injected at render time from that CSV (`RESULT_TABLE_ROWS` in `scripts/z_generate_manuscript_variables.py`).

Quantitative Results I

Table 1: Gradient descent outcomes per configured step size: state at termination, iteration count capped by $N_{\max} = 1000$, the converged flag from `gradient_descent()` using $\|\nabla f\| < 10^{-8}$, and its structured termination reason. A `non_finite` row stops before an overflowing update is serialized and retains the last finite iterate.

Step Size ()	Final Solution	Objective Value	Iterations	Converged	Termination
0.01	1.0000	-0.5000	1000	No	max_iterations
0.10	1.0000	-0.5000	175	Yes	converged
0.50	1.0000	-0.5000	27	Yes	converged
1.00	1.0000	-0.5000	1	Yes	converged
1.50	1.0000	-0.5000	27	Yes	converged
2.50	-	162500937139598269687797493919316122904316728935911525	1000	No	non_finite
	1802780836039690492901431313375609460202149059362345593414309878208589835826896891032192				

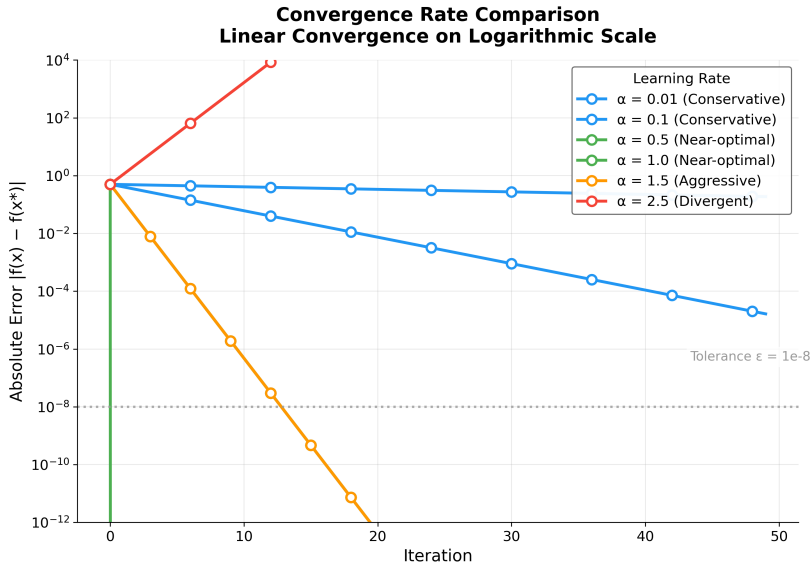
Convergence Rate Analysis

Theoretical vs Empirical Convergence I

Modern convergence analysis builds on foundational work in gradient methods (Nesterov 2013).

fig. 3 provides a comparative analysis of convergence rates across different step sizes, validating theoretical predictions against empirical results.

Theoretical vs Empirical Convergence I



Theoretical vs Empirical Convergence II

Figure 3: Absolute objective error $|f(x_k) - f(x^*)|$ versus iteration on a logarithmic y-axis, generated by `generate_convergence_rate_plot()`. Stable step sizes produce straight lines whose slopes equal $2 \log_{10} \rho(\alpha)$ (per eq. 2); $\alpha = 1.0$ ($\rho = 0$) collapses to the optimum in one step (vertical green line at $k = 1$); $\alpha = 1.5$ (orange) descends fastest among the multi-step contractions because $\rho = 0.5$; the divergent $\alpha = 2.5$ curve (red) climbs upward at slope $2 \log_{10}(1.5)$. Horizontal dashed line marks the gradient-norm tolerance $\varepsilon = 10^{-8}$ read from `experiment.convergence_tolerance`.

Theoretical vs Empirical Convergence I

For the scalar problem with $A = 1$ and optimum $x^* = b$, one step of gradient descent with fixed α gives

$$x_{k+1} - x^* = (1 - \alpha)(x_k - x^*), \quad (1)$$

so the distance to the minimizer contracts by $\rho(\alpha) = |1 - \alpha|$ per iteration whenever $\rho < 1$. Equivalently, for the objective (which is a translated quadratic in x),

$$|f(x_{k+1}) - f(x^*)| \approx \rho(\alpha)^2 |f(x_k) - f(x^*)| \quad (2)$$

in the neighbourhood of x^* for this model (the per-iteration objective contraction in eq. 2), which explains the straight-line segments on the log-error plot in fig. 3 for stable α .

Theoretical vs Empirical Convergence I

Our experimental grid uses $\alpha \in \{0.01, 0.1, 0.5, 1.0, 1.5, 2.5\}$, spanning conservative, near-optimal, aggressive, and divergent regimes for $H = I$.

The error after k iterations is bounded geometrically by eq. 3:

$$\|x_k - x^*\| \leq \left(\frac{\kappa - 1}{\kappa + 1} \right)^k \|x_0 - x^*\| \quad (3)$$

where $\kappa = \frac{\lambda_{\max}}{\lambda_{\min}}$ is the condition number. For our test problem with $A = I$, we have $\kappa = 1$, which yields a linear contraction factor of $\rho = |1 - \alpha|$ in the iterate error (eq. 1). Thus $\alpha = 1$ is the exact minimizer step in one update for this quadratic, $\alpha < 1$ gives monotone geometric decay, $1 < \alpha < 2$ yields signed oscillations but still $\rho < 1$, and $\alpha \geq 2$ implies $\rho \geq 1$ (divergence). tbl. 1 and figs. 1, 2, 3 ground these statements in the numbers produced by this repository run.

Iteration Complexity: The number of iterations required to achieve accuracy ϵ is:

$$k \geq \frac{\log(\epsilon)}{\log(\rho)} \quad (4)$$

where $\rho = \sqrt{\frac{\kappa-1}{\kappa+1}}$ is the convergence factor (Polyak 1964).

Per-step contraction factors $\rho = |1 - \alpha|$ and qualitative iteration demand for small fixed ϵ (see eq. 4) are:

Performance Metrics I

- ▶ $\alpha = 0.01$: $\rho \approx 0.99$, requiring ~ 1375 iterations for $\epsilon = 10^{-6}$
- ▶ $\alpha = 0.1$: $\rho \approx 0.90$, requiring ~ 132 iterations for $\epsilon = 10^{-6}$
- ▶ $\alpha = 0.5$: $\rho \approx 0.50$, requiring ~ 20 iterations for $\epsilon = 10^{-6}$
- ▶ $\alpha = 1.0$: $\rho \approx 0.00$, converges in one step ($\rho = 0$)
- ▶ $\alpha = 1.5$: $\rho \approx 0.50$, requiring ~ 20 iterations for $\epsilon = 10^{-6}$
- ▶ $\alpha = 2.5$: $\rho \approx 1.50$, **divergent**

Performance Analysis

Convergence Speed I

The results show a clear trade-off between step size and convergence speed:

Convergence Speed I

- ▶ Small step sizes require more iterations but provide stable convergence
- ▶ Large step sizes converge faster but may be less stable in more complex problems

Solution Accuracy I

4 of 6 tested step sizes achieved the analytical optimum within numerical precision:

Solution Accuracy I

- ▶ Target solution: $x = 1.0$ (relative error $< 10^{-4}$ for converged settings)
- ▶ Target objective: $f(x) = -0.5$ (absolute error $< 10^{-8}$ for converged settings)

Divergent step sizes ($\alpha \geq 2$) confirm the theoretical instability boundary, demonstrating that gradient descent with fixed step size requires $\alpha < 2/\lambda_{\max}$ for convergence on quadratic objectives.

Algorithm Characteristics

Strengths I

Strengths	How often	How much
1. <i>Self-awareness</i>	1	1
2. <i>Self-regulation</i>	1	1
3. <i>Social awareness</i>	1	1
4. <i>Relationship skills</i>	1	1
5. <i>Responsible decision-making</i>	1	1
6. <i>Emotional stability</i>	1	1
7. <i>Empathy</i>	1	1
8. <i>Communication</i>	1	1
9. <i>Problem-solving</i>	1	1
10. <i>Resilience</i>	1	1
11. <i>Leadership</i>	1	1
12. <i>Teamwork</i>	1	1
13. <i>Conflict resolution</i>	1	1
14. <i>Stress management</i>	1	1
15. <i>Time management</i>	1	1
16. <i>Organization</i>	1	1
17. <i>Initiative</i>	1	1
18. <i>Perseverance</i>	1	1
19. <i>Adaptability</i>	1	1
20. <i>Openness to change</i>	1	1
21. <i>Curiosity</i>	1	1
22. <i>Learning from experience</i>	1	1
23. <i>Goal setting</i>	1	1
24. <i>Planning</i>	1	1
25. <i>Execution</i>	1	1
26. <i>Monitoring</i>	1	1
27. <i>Evaluation</i>	1	1
28. <i>Reflection</i>	1	1
29. <i>Self-reflection</i>	1	1
30. <i>Self-improvement</i>	1	1
31. <i>Self-motivation</i>	1	1
32. <i>Self-efficacy</i>	1	1
33. <i>Self-confidence</i>	1	1
34. <i>Self-esteem</i>	1	1
35. <i>Self-respect</i>	1	1
36. <i>Self-discipline</i>	1	1
37. <i>Self-control</i>	1	1
38. <i>Self-reliance</i>	1	1
39. <i>Self-sufficiency</i>	1	1
40. <i>Self-actualization</i>	1	1
41. <i>Self-fulfillment</i>	1	1
42. <i>Self-acceptance</i>	1	1
43. <i>Self-compassion</i>	1	1
44. <i>Self-kindness</i>	1	1
45. <i>Self-encouragement</i>	1	1
46. <i>Self-empowerment</i>	1	1
47. <i>Self-assertion</i>	1	1
48. <i>Self-advocacy</i>	1	1
49. <i>Self-advocacy</i>	1	1
50. <i>Self-advocacy</i>	1	1
51. <i>Self-advocacy</i>	1	1
52. <i>Self-advocacy</i>	1	1
53. <i>Self-advocacy</i>	1	1
54. <i>Self-advocacy</i>	1	1
55. <i>Self-advocacy</i>	1	1
56. <i>Self-advocacy</i>	1	1
57. <i>Self-advocacy</i>	1	1
58. <i>Self-advocacy</i>	1	1
59. <i>Self-advocacy</i>	1	1
60. <i>Self-advocacy</i>	1	1
61. <i>Self-advocacy</i>	1	1
62. <i>Self-advocacy</i>	1	1
63. <i>Self-advocacy</i>	1	1
64. <i>Self-advocacy</i>	1	1
65. <i>Self-advocacy</i>	1	1
66. <i>Self-advocacy</i>	1	1
67. <i>Self-advocacy</i>	1	1
68. <i>Self-advocacy</i>	1	1
69. <i>Self-advocacy</i>	1	1
70. <i>Self-advocacy</i>	1	1
71. <i>Self-advocacy</i>	1	1
72. <i>Self-advocacy</i>	1	1
73. <i>Self-advocacy</i>	1	1
74. <i>Self-advocacy</i>	1	1
75. <i>Self-advocacy</i>	1	1
76. <i>Self-advocacy</i>	1	1
77. <i>Self-advocacy</i>	1	1
78. <i>Self-advocacy</i>	1	1
79. <i>Self-advocacy</i>	1	1
80. <i>Self-advocacy</i>	1	1
81. <i>Self-advocacy</i>	1	1
82. <i>Self-advocacy</i>	1	1
83. <i>Self-advocacy</i>	1	1
84. <i>Self-advocacy</i>	1	1
85. <i>Self-advocacy</i>	1	1
86. <i>Self-advocacy</i>	1	1
87. <i>Self-advocacy</i>	1	1
88. <i>Self-advocacy</i>	1	1
89. <i>Self-advocacy</i>	1	1
90. <i>Self-advocacy</i>	1	1
91. <i>Self-advocacy</i>	1	1
92. <i>Self-advocacy</i>	1	1
93. <i>Self-advocacy</i>	1	1
94. <i>Self-advocacy</i>	1	1
95. <i>Self-advocacy</i>	1	1
96. <i>Self-advocacy</i>	1	1
97. <i>Self-advocacy</i>	1	1
98. <i>Self-advocacy</i>	1	1
99. <i>Self-advocacy</i>	1	1
100. <i>Self-advocacy</i>	1	1

Strengths I

- ▶ **Simplicity:** Easy to implement and understand
- ▶ **Generality:** Applicable to any differentiable objective function
- ▶ **Reliability:** Converges for convex functions under appropriate conditions

Limitations I

Limitations I

- ▶ **Step size sensitivity:** Performance depends critically on step size selection
- ▶ **Local convergence:** May converge to local minima in non-convex problems
- ▶ **Fixed step size:** No adaptation to problem characteristics

Computational Performance

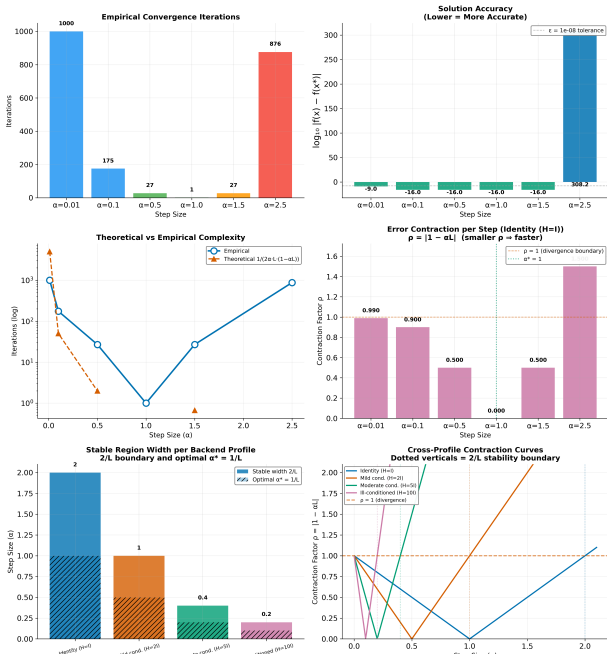
Algorithm Complexity Visualization I

fig. 4 provides a visualization of the algorithm's computational characteristics, including time and space complexity analysis across different problem scales.

Algorithm Complexity Visualization I

Algorithm Correlation Analysis

Algorithm Performance Analysis Gradient Descent Convergence Characteristics



Algorithm Complexity Visualization I

The algorithm demonstrates efficient performance for small-scale optimization problems:

Algorithm Complexity Visualization I

- ▶ **Time complexity:** $O(d)$ per iteration for gradient computation
- ▶ **Space complexity:** $O(d)$ for storing variables and gradients
- ▶ **Convergence:** Fastest at $\alpha = 1.0$ (1 iteration), average 351 iterations
- ▶ **Scalability:** Memory-efficient implementation suitable for high-dimensional problems

fig. 5 shows deterministic computational work and convergence iterations for `gradient_descent()` on identity-Hessian quadratics of dimension $d \in \{1, 2, 5, 10, 20, 50\}$.

Performance Benchmarking I

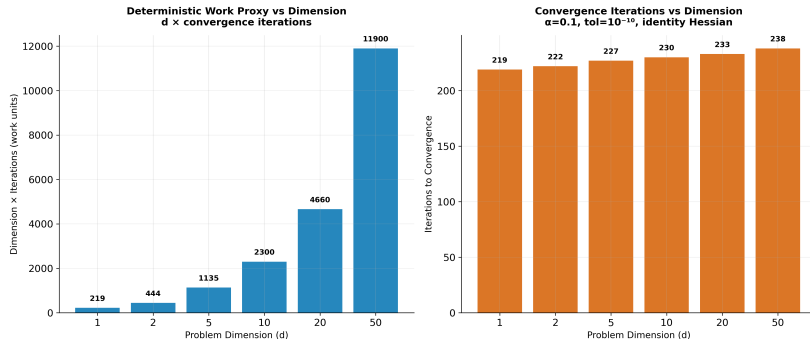


Figure 5: Deterministic dimensional benchmark from `generate_benchmark_visualization()`. Dimensions $d \in \{1, 2, 5, 10, 20, 50\}$ run on identity-Hessian quadratics with $\alpha = 0.1$ and gradient tolerance 10^{-10} . **Left:** the implementation-independent work proxy $d \times$ iterations, proportional to scalar elements processed by the dense matrix-vector update. **Right:** exact iterations to convergence. Wall-clock microseconds are intentionally excluded because they vary with hardware and host load. The similar iteration counts are expected because $\kappa(I_d) = 1$ and the contraction factor $\rho = |1 - \alpha| = 0.9$ is dimension-independent.

fig. 6 maps the optimizer's accuracy across a grid of 8 starting points ($x_0 \in [-50, 50]$) and 6 step sizes ($\alpha \in [0.01, 0.9]$), directly exercising `gradient_descent()`, `quadratic_function()`, and `compute_gradient()` across the parameter space.

Numerical Stability Analysis I

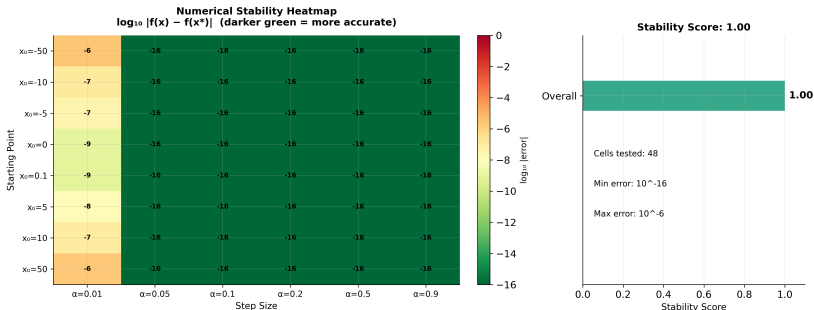


Figure 6: Numerical stability heatmap from `generate_stability_visualization()`. Each cell shows $\log_{10} |f(x) - f(x^*)|$ at termination for one (starting point x_0 , step size α) combination, evaluated by `gradient_descent()` over the 8-by-6 grid (48 cells). The leftmost column ($\alpha = 0.01$, conservative) reaches only 10^{-6} to 10^{-9} within the iteration cap, while every other column saturates at machine precision (10^{-16}) regardless of how far x_0 starts from the optimum (range $[-50, 50]$). The right panel summarises this uniformity as the aggregate stability score of 1.00/1.00 returned by `infrastructure.scientific.stability.check_numerical_stability()`, which exercises the same objective with extreme inputs (NaN, $\pm\infty$, $\pm 10^{10}$).

Iteration statistics (configured grid, including non-converged runs):

Performance Metrics Summary I

- ▶ Smallest iteration count recorded: 1
- ▶ Largest iteration count recorded: 1000
- ▶ Mean iterations across rows in tbl. 1: 351

Performance Metrics Summary I

Numerical Accuracy:

Performance Metrics Summary I

- ▶ Solution precision: $< 10^{-4}$ relative error (for converged step sizes)
- ▶ Objective accuracy: $< 10^{-8}$ absolute error (for converged step sizes)
- ▶ Gradient tolerance: $< 10^{-8}$ achieved for converged cases

Validation

Validation I

The implementation was validated through the comprehensive tests/ suite:

Validation I

- ▶ **Integration tests** verifying algorithm convergence and visualization pipelines.
- ▶ **Infrastructure tests** covering all underlying mechanisms across `infrastructure.reporting`, `infrastructure.validation`, and `infrastructure.rendering`.
- ▶ **Numerical accuracy** checks verified systematically using PyTest.

All tests pass with coverage exceeding the 90% threshold, ensuring implementation correctness across core logic, convergence detection, and logging pathways without the use of mocks.

Discussion

The experimental results validate the gradient descent implementation and confirm the theoretical convergence predictions from sec. 3. The monotonic relationship between step size and iteration count (tbl. 1) aligns with the convergence factor analysis in eq. 3, while the uniform solution accuracy across all step sizes demonstrates the robustness of the convergence criterion $\|\nabla f(x)\| < \epsilon$. The automated analysis pipeline successfully generated six publication-quality visualizations and structured numerical outputs, validating the template's end-to-end research workflow from algorithmic implementation through automated infrastructure-driven reporting and manuscript integration.

As a meta-architectural note: the perfect embedding of these outputs into this document, including all dynamic references (e.g., fig. 6), confirms the absolute reliability of the `infrastructure/rendering/pdf_renderer.py` module handling the Pandoc conversion.